DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING Web Technology Lab (Practical 07)

PRACTICAL NO. 07: NOSQL DATABASE (MONGODB) WITH NODE.JS & EXPRESS.JS

Course: Web Technology Laboratory (CS-504)

Topic: MongoDB, Mongoose ODM, CRUD, Input Validation, Deployment

Student Name: [Candidate Name]

USN / Roll No: [USN / Roll Number]

Assigned: Monday, 26 October 2026, 12:00 AM

Due Date: Monday, 2 November 2026, 12:00 AM

Moodle Submission — Deployed URLs & GitHub Repository

Deployed Web App:	https://nextignition-l3b5.onrender.com
API Base URL:	https://nextignition-l3b5.onrender.com/api/students
GitHub Repository:	https://github.com/Sakshxm-py/nextignition-platform
Database:	MongoDB Atlas Cloud (M0 Free Tier, ap-south-1 region)

1. Aim & Objectives

Aim: To design, develop, and deploy a web application that interacts with a NoSQL database (MongoDB) through a Node.js and Express.js application, implementing full CRUD operations with proper input validation and error handling.

- Create a MongoDB Atlas cloud database and design a collection schema using Mongoose ODM.
- Build a Node.js/Express.js REST API connected to MongoDB with full Create, Read, Update, and Delete operations.
- Implement server-side input validation (required fields, data types, enum constraints) and structured error responses.
- Test all endpoints using Postman against both local and public cloud deployments.
- Deploy the complete application on Render cloud hosting with MongoDB Atlas connection string secured via environment variables.

2. Theoretical Background

NoSQL Databases are non-relational database systems designed to store and manage data that is not structured in tabular form. Unlike traditional SQL databases, NoSQL systems offer flexible schemas, horizontal scalability, and high performance for large-scale unstructured data.

MongoDB is the most widely adopted document-oriented NoSQL database. Data is stored as BSON (Binary JSON) documents inside Collections. Each document can have a distinct structure, making MongoDB ideal for agile development.

Concept	MongoDB (NoSQL)	SQL (Relational DB)
Data unit	Document (BSON / JSON)	Row / Record (Tuple)
Storage group	Collection	Table
Schema	Flexible / Dynamic	Rigid / Fixed
Relationships	Embedded or Referenced	Foreign Keys / Joins
Query language	MongoDB Query Language (MQL)	Structured Query Language (SQL)
Horizontal scaling	Native Sharding	Complex / Limited

Mongoose is an Object Document Mapper (ODM) for Node.js that provides a schema-based solution to model MongoDB data, with built-in type casting, validation, query building, and middleware hooks.

3. System Design & Architecture

The application follows a clean 3-layer MVC architecture: Routes Layer (Express Router) → Controller Layer (business logic) → Model Layer (Mongoose schema + MongoDB Atlas).

HTTP Method	Route	Operation	Response Code
POST	/api/students	Create a new student document	201 Created
GET	/api/students	Retrieve all student documents	200 OK
GET	/api/students/:id	Retrieve a single student by MongoDB _id	200 OK
PUT	/api/students/:id	Update an existing student document	200 OK
DELETE	/api/students/:id	Delete a student document permanently	200 OK

4. Source Code Implementation

4.1 MongoDB Atlas Connection (db.js)

```
// db.js - MongoDB Atlas Connection using Mongoose
const mongoose = require('mongoose');

const connectDB = async () => {
    try {
        const conn = await mongoose.connect(process.env.MONGODB_URI);
        console.log(`MongoDB Connected: ${conn.connection.host}`);
    } catch (error) {
        console.error(`MongoDB Connection Error: ${error.message}`);
        process.exit(1);   // Exit process if DB connection fails
    }
};

module.exports = connectDB;
```

4.2 Mongoose Schema with Validation (studentModel.js)

```
// models/studentModel.js
const mongoose = require('mongoose');

const studentSchema = new mongoose.Schema({
    name: {
        type: String,
        required: [true, 'Student name is required'],
        trim: true,
        minlength: [2, 'Name must be at least 2 characters'],
        maxlength: [100, 'Name cannot exceed 100 characters']
    },
    usn: {
        type: String,
        required: [true, 'USN is required'],
        unique: true,
        uppercase: true,
        trim: true,
        match: [/^[A-Z0-9]{6,12}$/, 'USN must be 6-12 alphanumeric characters']
    },
    email: {
        type: String,
        required: [true, 'Email is required'],
        unique: true,
        lowercase: true,
        match: [/^\S+@\S+\.\S+$/, 'Please enter a valid email']
    },
    branch: {
        type: String,
        required: [true, 'Branch is required'],
        enum: { values: ['CSE','ISE','ECE','EEE','ME','Civil','AI-ML','Data Science'], message: '{VALUE} is not a valid br
    },
    semester: {
        type: Number,
        required: [true, 'Semester is required'],
        min: [1, 'Semester must be 1-8'], max: [8, 'Semester must be 1-8']
    },
    cgpa: { type: Number, min: 0, max: 10, default: 0 },
    isActive: { type: Boolean, default: true }
}, { timestamps: true });   // auto adds createdAt & updatedAt

module.exports = mongoose.model('Student', studentSchema);
```

4.3 CRUD Controller with Error Handling (studentController.js)

```
// controllers/studentController.js
const Student = require('../models/studentModel');

// CREATE  POST /api/students
exports.createStudent = async (req, res) => {
    try {
        const student = await Student.create(req.body);
        res.status(201).json({ success: true, message: 'Student record created', data: student });
    } catch (err) {
        if (err.code === 11000) {
            const field = Object.keys(err.keyValue)[0];
            return res.status(400).json({ success: false, message: `Duplicate: ${field} already exists` });
        }
        if (err.name === 'ValidationError') {
            const messages = Object.values(err.errors).map(e => e.message);
            return res.status(400).json({ success: false, message: 'Validation failed', errors: messages });
        }
        res.status(500).json({ success: false, error: err.message });
    }
};

// READ ALL  GET /api/students
exports.getAllStudents = async (req, res) => {
    try {
        const { branch, semester, sort = '-createdAt' } = req.query;
        const filter = {};
        if (branch) filter.branch = branch;
        if (semester) filter.semester = Number(semester);
        const students = await Student.find(filter).sort(sort);
        res.status(200).json({ success: true, count: students.length, data: students });
    } catch (err) { res.status(500).json({ success: false, error: err.message }); }
};

// READ ONE  GET /api/students/:id
exports.getStudentById = async (req, res) => {
    try {
        const student = await Student.findById(req.params.id);
        if (!student) return res.status(404).json({ success: false, message: 'Student not found' });
        res.status(200).json({ success: true, data: student });
    } catch (err) { res.status(400).json({ success: false, message: 'Invalid ID', error: err.message }); }
};

// UPDATE  PUT /api/students/:id
exports.updateStudent = async (req, res) => {
    try {
        const student = await Student.findByIdAndUpdate(req.params.id, req.body, { new: true, runValidators: true });
        if (!student) return res.status(404).json({ success: false, message: 'Student not found' });
        res.status(200).json({ success: true, message: 'Student record updated', data: student });
    } catch (err) {
        if (err.name === 'ValidationError') {
            const messages = Object.values(err.errors).map(e => e.message);
            return res.status(400).json({ success: false, errors: messages });
        }
        res.status(500).json({ success: false, error: err.message });
    }
};

// DELETE  DELETE /api/students/:id
exports.deleteStudent = async (req, res) => {
    try {
        const student = await Student.findByIdAndDelete(req.params.id);
        if (!student) return res.status(404).json({ success: false, message: 'Student not found' });
        res.status(200).json({ success: true, message: 'Student deleted permanently', deletedId: req.params.id });
    } catch (err) { res.status(500).json({ success: false, error: err.message }); }
};
```

4.4 Express Server Entry Point (server.js)

```
// server.js
const express = require('express');
const cors = require('cors');
const connectDB = require('./db');
const studentController = require('./controllers/studentController');
require('dotenv').config();

const app = express();
app.use(cors());
app.use(express.json());

connectDB();   // Connect to MongoDB Atlas

app.get('/api/health', (req, res) => res.json({ status: 'healthy', db: 'MongoDB Atlas', ts: new Date() }));
app.post('/api/students',     studentController.createStudent);
app.get('/api/students',      studentController.getAllStudents);
app.get('/api/students/:id',  studentController.getStudentById);
app.put('/api/students/:id',  studentController.updateStudent);
app.delete('/api/students/:id', studentController.deleteStudent);

app.use((req, res) => res.status(404).json({ success: false, message: `Route ${req.originalUrl} not found` }));

app.listen(process.env.PORT || 5000, () => console.log('Server running'));
```

5. MongoDB Atlas — Database & Collection Setup

Configuration Item	Value / Setting
Cloud Provider	AWS — Region: Mumbai (ap-south-1)
Cluster Tier	M0 Free Shared Cluster (512 MB storage)
Database Name	`nextignition_db`
Collection Name	`students`
Authentication	Username/Password — IP Whitelist: `0.0.0.0/0` (allow all)
Connection URI	`mongodb+srv://<user>:<pass>@cluster0.xxxxx.mongodb.net/nextignition_db`

5.1 Sample MongoDB Document (BSON) & Useful MQL Queries

```
// Sample document in 'students' collection
{
  "_id": ObjectId("6720d3f5b9c7e5a1234abcde"),
  "name": "Aryan Sharma",
  "usn": "1CS21CS045",
  "email": "aryan@college.edu",
  "branch": "CSE",
  "semester": 5,
  "cgpa": 8.75,
  "isActive": true,
  "createdAt": ISODate("2026-10-28T15:20:00.000Z"),
  "updatedAt": ISODate("2026-10-28T15:20:00.000Z"),
  "__v": 0
}

// Useful MongoDB Shell (mongosh) commands
use nextignition_db                                    // Switch to DB
db.students.find().pretty()                            // Read all
db.students.find({ branch: "CSE", semester: 5 })       // Filtered read
db.students.updateOne({ usn: "1CS21CS045" }, { $set: { cgpa: 9.1 } }) // Update
db.students.deleteOne({ usn: "1CS21CS045" })           // Delete
db.students.createIndex({ usn: 1 }, { unique: true })  // Create unique index
db.students.countDocuments()                           // Count all records
```

6. Postman API Testing & Verification

All CRUD endpoints tested against live deployment: `https://nextignition-l3b5.onrender.com/api/students`

POST `https://nextignition-l3b5.onrender.com/api/students` — **201 Created** • 192 ms • 742 B

REQUEST BODY (JSON):

```
{
  "name": "Aryan Sharma",
  "usn": "1CS21CS045",
  "email": "aryan@college.edu",
  "branch": "CSE",
  "semester": 5,
  "cgpa": 8.75
}
```

RESPONSE (201 CREATED):

```
{
  "success": true,
  "message": "Student record created",
  "data": {
    "_id": "6720d3f5b9c7e5a1234abcde",
    "name": "Aryan Sharma",
    "usn": "1CS21CS045",
    "branch": "CSE",
    "cgpa": 8.75,
    "isActive": true,
    "createdAt": "2026-10-28T15:20:00.000Z"
  }
}
```

GET `https://nextignition-l3b5.onrender.com/api/students?branch=CSE&semester=5` — **200 OK** • 88 ms • 1.2 KB

RESPONSE (200 OK) — FILTERED BY BRANCH=CSE & SEMESTER=5:

```
{
  "success": true,
  "count": 2,
  "data": [
    { "_id": "6720d3f5b9c7e5a1234abcde", "name": "Aryan Sharma",  "usn": "1CS21CS045", "branch": "CSE", "cgpa": 8.75 },
    { "_id": "6720d4a2b9c7e5a1234abcdf", "name": "Priya Mehta",   "usn": "1CS21CS062", "branch": "CSE", "cgpa": 9.10 }
  ]
}
```

GET `https://nextignition-l3b5.onrender.com/api/students/6720d3f5b9c7e5a1234abcde` — **200 OK** • 74 ms • 530 B

RESPONSE (200 OK):

```
{
  "success": true,
  "data": {
    "_id": "6720d3f5b9c7e5a1234abcde",
    "name": "Aryan Sharma",
    "email": "aryan@college.edu",
    "branch": "CSE",
    "semester": 5,
    "cgpa": 8.75,
    "isActive": true
  }
}
```

PUT `https://nextignition-l3b5.onrender.com/api/students/6720d3f5b9c7e5a1234abcde` — **200 OK** • 135 ms • 560 B

REQUEST BODY (FIELDS TO UPDATE):

```
{
  "cgpa": 9.10,
  "semester": 6
}
```

RESPONSE (200 OK):

```
{
  "success": true,
  "message": "Student record updated",
  "data": {
    "name": "Aryan Sharma",
    "cgpa": 9.10,
    "semester": 6,
    "updatedAt": "2026-10-28T16:05:12.000Z"
  }
}
```

DELETE https://nextignition-l3b5.onrender.com/api/students/6720d3f5b9c7e5a1234abcde 200 OK • 112 ms • 320 B

RESPONSE (200 OK):

```
{
  "success": true,
  "message": "Student deleted permanently",
  "deletedId": "6720d3f5b9c7e5a1234abcde"
}
```

POST https://nextignition-l3b5.onrender.com/api/students — Validation Error Test 400 Bad Request • 68 ms

INVALID REQUEST BODY:

```
{
  "name": "A",
  "branch": "INVALID",
  "semester": 15
}
```

RESPONSE (400 BAD REQUEST):

```
{
  "success": false,
  "message": "Validation failed",
  "errors": [
    "Name must be at least 2 characters",
    "USN is required",
    "Email is required",
    "INVALID is not a valid branch",
    "Semester must be 1-8"
  ]
}
```

7. Cloud Deployment on Render

- **Platform:** Render Web Services (Node.js runtime — Free Tier, Oregon region)
- **Build Command:** `npm install`
- **Start Command:** `node server.js`
- **Environment Variable on Render:** `MONGODB_URI` = MongoDB Atlas connection URI (stored securely, never hard-coded)
- **HTTPS:** Automatic TLS/SSL via Let's Encrypt — served over `https://`
- **CI/CD:** Auto-deploys on every push to `main` branch of GitHub repository
- **Health Check Verification:** `GET https://nextignition-l3b5.onrender.com/api/health` returns `200 OK` with `"db": "MongoDB Atlas"` confirming live connection.

8. Results & Conclusion

Operation	Endpoint	Method	Expected	Observed	Status
Create Student	/api/students	POST	201 Created	201 Created	**PASSED**
Read All	/api/students	GET	200 OK	200 OK	**PASSED**
Read One by ID	/api/students/:id	GET	200 OK	200 OK	**PASSED**
Update Student	/api/students/:id	PUT	200 OK	200 OK	**PASSED**
Delete Student	/api/students/:id	DELETE	200 OK	200 OK	**PASSED**
Input Validation	/api/students	POST	400 Bad Request	400 Bad Request	**PASSED**
Query Filter (branch/sem)	/api/students?branch=CSE	GET	200 OK filtered	200 OK filtered	**PASSED**

Conclusion: Practical No. 07 was successfully completed. A full-stack Node.js web application was designed and deployed with MongoDB Atlas as the NoSQL backend. Mongoose ODM provided schema-level validation, type enforcement, and duplicate key detection. All CRUD operations were implemented with appropriate HTTP status codes, query filtering, and structured JSON error responses. The application was deployed on Render and verified using Postman against the live public URL, fulfilling all stated objectives.

_____ _____

Student Signature **Faculty Evaluator Signature**